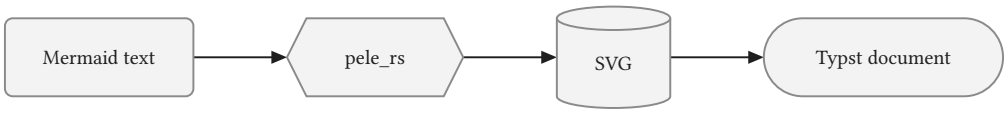
pele-rs

Mermaid diagrams in Typst

Version 0.3.0 · github.com/vini-fda/pele_typst



Contents

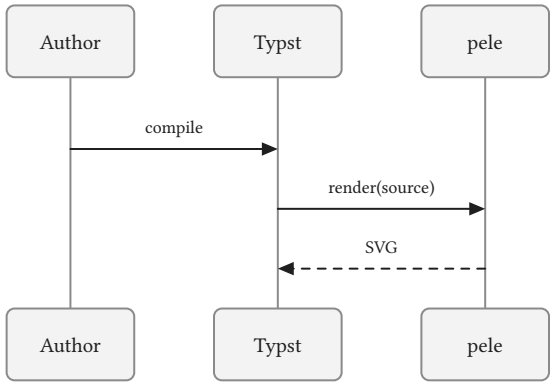
1 Overview 2
2 Getting started 3
2.1 Usage 3
2.2 Code blocks 3
2.3 Defaults for a whole document 4
3 Basics 5
3.1 Size 5
3.2 Fonts 5
3.3 Themes 5
3.4 Configuration 6
3.5 Dates and times 7
3.6 Icons and math 7
3.7 Accessibility 8
3.8 Errors 8
4 API reference 9
4.1 `mermaid` 9
5 Diagram types 11
5.1 Flow and charts 11
5.2 Structured layouts 15
5.3 Graphs 21
5.4 Specialized engines 26
6 Compatibility 30

1 Overview

pele-rs draws Mermaid diagrams in Typst documents. You write the diagram as text, and it is laid out and drawn while your document compiles: no browser, no Node.js, no image files to keep in sync.

It is built on `pele_rs`, a Rust port of pele, the fast Mermaid renderer by Obsidian, compiled to a WebAssembly plugin. This version implements all 33 diagram types of pele `1.12.0`, and its output matches the original's byte for byte. It works in the Typst CLI and in the web app alike. pele-rs is an independent port, not an official Obsidian or Mermaid project.

```
#mermaid("
sequenceDiagram
  Author->>Typst: compile
  Typst->>pele: render(source)
  pele-->>Typst: SVG
")
```



2 Getting started

2.1 Usage

Import the `mermaid` function and call it with the diagram's source:

```
#import "@preview/pele-rs:0.3.0": mermaid

#mermaid("flowchart LR\n  A --> B --> C")
```

The result is an image, so it can go wherever an image can: in a `figure`, in a `grid`, or inside `align`, `scale` or `rotate`.

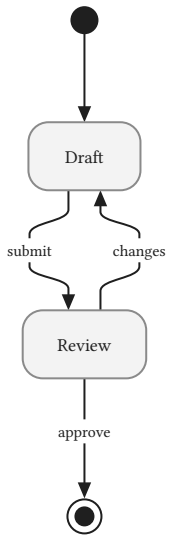
```
#figure(
  mermaid("flowchart LR\n  A --> B --> C"),
  caption: [A small flowchart.],
)
```



Figure 1: A small flowchart.

2.2 Code blocks

Longer diagrams are easier to write in a fenced code block. A show rule turns every block tagged `mermaid` into a diagram:




````
#show raw.where(lang: "mermaid"): it => mermaid(
  it, font: "Libertinus Serif",
)

```mermaid
stateDiagram-v2
  [*] --> Draft
  Draft --> Review: submit
  Review --> Draft: changes
  Review --> [*]: approve
```
````

The text of a code block is set in a monospace font, which `mermaid` would otherwise take as the label font. That is why the rule above names the `font` to use.

2.3 Defaults for a whole document

`mermaid` is a function, not an element, so it cannot be configured with a `set` rule. To use the same options everywhere, define your own version with `.with`:

```
#import "@preview/pele-rs:0.3.0": mermaid
#let mermaid = mermaid.with(
  font: "Libertinus Serif",
  theme: (accent: rgb("#2f5fb3"), radius: 6pt),
  now: 1768478400000,
)
#show raw.where(lang: "mermaid"): mermaid
```

3 Basics

3.1 Size

A diagram is drawn at its natural size, with labels at the size of the surrounding text. Like `image`, `mermaid` takes a `width` and a `height` to scale it, and a `fit` used when both are given. Extra room is added around the drawing with `padding`.

```
#mermaid(width: 60%, padding: 0pt,
  "flowchart LR\n  A --> B --> C")
```



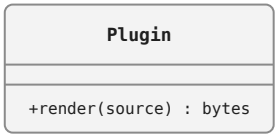

```
#mermaid(height: 4em,
  "flowchart LR\n  A --> B --> C")
```



3.2 Fonts

Labels use the surrounding text's font and size unless `font` and `font-size` say otherwise. Some diagrams draw parts of their text in a monospace font, given by `mono-font`; which parts is chosen with `monospace`.

```
#mermaid(font: "DejaVu Sans Mono", font-size:
9pt,
  monospace: ("members",), "
classDiagram
  class Plugin {
    +render(source) bytes
  }
")
```



Labels are measured with the character widths of Arial, as pele does outside a browser, and drawn in the font you choose. Fonts of about the same width, or narrower, fit their boxes; a much wider font can overflow them.

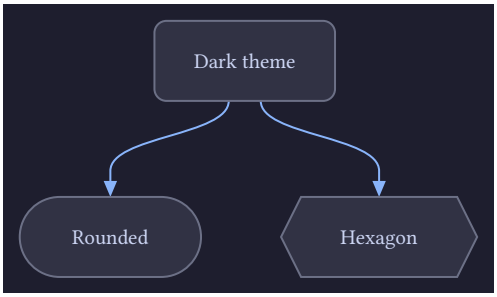
3.3 Themes

Colors and corner radii come from pele's CSS variables. Pass them in `theme`, by their name without the `--pele-` prefix. Values may be Typst colors, lengths or strings.

Key	Default	Used for
`bg`	`#fff`	Background, painted only when set
`fg`	`#1f1f1f`	Text
`muted`	`#6e6e6e`	Secondary text, such as notes and axes
`line`	`#1f1f1f`	Edges and arrows
`surface`	`#f3f3f3`	Node fill
`surface-alt`	`#e6e6e6`	Alternate fill, such as subgraphs and headers
`border`	`#8a8a8a`	Node outlines
`accent`	`#5b7bd5`	Highlights, such as active tasks and selections
`radius`	`4px`	Corner radius of nodes
`fg-on-light`	`#1f1f1f`	Text on light, colored fills
`fg-on-dark`	`#fff`	Text on dark, colored fills
`critical`	`#e45756`	Critical items, such as Gantt's critical tasks
`bold-weight`	`bold`	Weight of bold text
`title-weight`	`bold`	Weight of titles
`heading-weight`	`bold`	Weight of headings

Setting `bg` also paints the background behind the diagram; `background` turns this on or off explicitly.

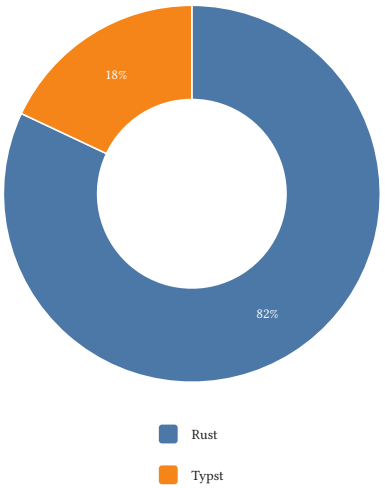
```
#mermaid(theme: (
  bg: rgb("#1e1e2e"), fg: rgb("#cdd6f4"),
  line: rgb("#89b4fa"), surface:
rgb("#313244"),
  border: rgb("#6c7086"), radius: 6pt,
), "
flowchart TD
  A[Dark theme] --> B([Rounded])
  A --> C{{Hexagon}}
")
```

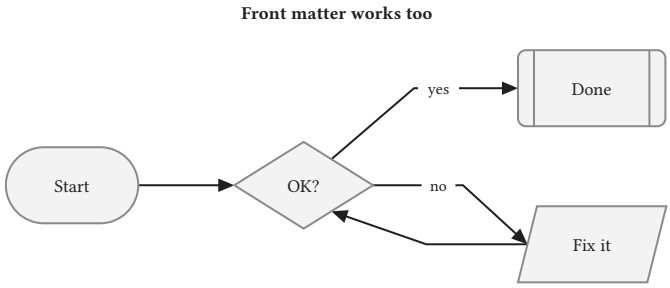


3.4 Configuration

Mermaid's configuration options can be set in three places, from lowest to highest precedence: the `config` argument, front matter at the top of the source, and `%%{init: ...}%%` directives.

```
#mermaid(width: 70%,
  config: (pie: (donutHole: 0.5,
    legendPosition: "bottom")),
  "pie\n  \"Rust\" : 82\n  \"Typst\" : 18",
)
```



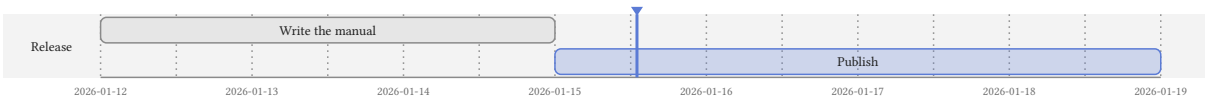



```
#show raw.where(lang: "mermaid"): it => mermaid(it, font: "Libertinus Serif")

```mermaid
---
title: Front matter works too
config:
  flowchart:
    curve: linear
---
graph LR
  start([Start]) --> check{OK?}
  check -- yes --> done[[Done]]
  check -- no --> fix[/Fix it/] --> check
```
```

3.5 Dates and times

Gantt charts mark today's date and resolve relative dates such as `after a` against the current time. A Typst document has no clock, so these charts require `now`, a Unix timestamp in milliseconds; this also makes every build produce the same chart. Calendar arithmetic uses `time-zone`, an IANA name defaulting to `"UTC"`, including daylight-saving transitions.




```
#mermaid(now: 1768478400000, time-zone: "Europe/Paris", "
gantt
  dateFormat YYYY-MM-DD
  section Release
    Write the manual :done, m, 2026-01-12, 3d
    Publish          :active, after m, 4d
")
```

To use the date of the build, compute `now` from `datetime.today()`. In the CLI, setting the `SOURCE_DATE_EPOCH` environment variable pins that date for reproducible builds.

```
#let epoch = datetime(year: 1970, month: 1, day: 1)
#let now = int((datetime.today() - epoch).seconds()) * 1000
```

3.6 Icons and math

pele draws icons written as `fa:fa-name` or `prefix:name` in labels, and math written between `$$`, with assets you supply. `icons` maps each icon's name to SVG markup drawn in a 24×24 viewport. `math` maps each formula's

TeX source to its SVG markup and its size in pixels. Icons and formulas without an asset fall back to pele's default behavior: formulas are drawn as text.

```
#mermaid("
flowchart LR
  A[\"fa:fa-star Starred\"] --> B[\"$$x^2$$\"]
", icons: (
  "fa:fa-star": "<path d='M12 2l3 7h7l-5.5 4.5
2 7.5-6.5-4.5-6.5 4.5 2-7.5L2 9h7z'
fill='currentColor'/>",
), math: (
  "x^2": (svg: "<svg viewBox='0 0 24 18'><text
y='15' font-size='15' font-style='italic'>x²</
text></svg>", width: 24, height: 18),
))
```



No icon set or math typesetter is bundled. SVG for math can be produced ahead of time with a tool such as MathJax; HTML math is not supported.

3.7 Accessibility

Pass a description of the diagram in `alt`. It becomes the image's alternative text in the PDF, which screen readers announce.

```
#mermaid(alt: "Typst sends the source to pele, which returns SVG.", "...")
```

3.8 Errors

Invalid source stops the compilation with pele's message, which gives the line and what was expected:

```
error: plugin errored with: Parse error on line 2:
 A -->
------^
Expecting 'NODE_STRING', got 'EOF'
```

4 API reference

4.1 `mermaid`

```
mermaid(
  source str raw,
  width: auto relative,
  height: auto relative,
  fit: str,
  font: auto str array,
  font-size: auto length,
  mono-font: str array,
  monospace: none array,
  now: none int float,
  time-zone: str,
  icons: dictionary,
  math: dictionary,
  padding: length,
  theme: dictionary,
  background: auto bool,
  config: dictionary,
  alt: none str
) → content
```

Draws a Mermaid diagram.

The diagram type is detected from the first line of the source, as in Mermaid. Front matter and `%%{init: ...}%%` directives are honored. Syntax errors stop the compilation with pele's message, which names the line and the expected token.

The result is an `image`, so it can be placed in a `figure`, scaled, aligned or rotated like any other image.

Parameters

`source` `str` `raw`

The Mermaid text, or a raw block holding it. Raw blocks make it easy to render every ```` ```mermaid ```` block with a show rule.

`width` `auto` `relative` — Default: `auto`

Width of the image. By default, the diagram's natural width.

`height` `auto` `relative` — Default: `auto`

Height of the image. By default, the diagram's natural height.

`fit` `str` — Default: `"contain"`

How the diagram fills a box given both a `width` and a `height`: `"contain"`, `"cover"` or `"stretch"`, as for `image`.

`font` `auto` `str` `array` — Default: `auto`

Font family for labels. By default, the surrounding text's font.

`font-size` `auto` `length` — Default: `auto`

Size of node labels. By default, the surrounding text's size.

`mono-font` `str` `array` Default: `"DejaVu Sans Mono"`

Font family for text pele draws in monospace, such as class members and file trees.

`monospace` `none` `array` Default: `none`

Which parts are drawn in `mono-font`, from `"members"`, `"attributes"`, `"files"` and `"messages"`. `none` keeps pele's defaults.

`now` `none` `int` `float` Default: `none`

The current time, as a Unix timestamp in milliseconds. Gantt charts require it, so that their today marker and relative dates are reproducible.

`time-zone` `str` Default: `"UTC"`

IANA time zone for Gantt calendar arithmetic, such as `"Europe/Paris"`. Daylight-saving transitions are respected.

`icons` `dictionary` Default: `(:)`

Icons by decoded name, such as `"fa:fa-star"`, mapped to SVG markup drawn in a 24×24 viewport. Missing icons fall back to pele's default behavior.

`math` `dictionary` Default: `(:)`

Math by TeX source, mapped to `(svg: str, width: float, height: float)` in pixels. Labels written as `$$...$$` use it; unknown formulas are drawn as text.

`padding` `length` Default: `8pt`

Room around the drawing.

`theme` `dictionary` Default: `(:)`

Colors and sizes by the short name of pele's CSS variables, such as `(fg: black, bg: white, accent: blue, surface: luma(240), radius: 4pt)`. Values may be colors, lengths or strings. Keys starting with `--` are passed as raw CSS variables.

`background` `auto` `bool` Default: `auto`

Whether to paint the theme's `bg` behind the diagram. By default, only when the theme sets `bg`.

`config` `dictionary` Default: `(:)`

Mermaid configuration, as in front matter's `config:`, such as `(flowchart: (curve: "linear"))` or `(pie: (donutHole: 0.5))`. The source's own front matter and directives take precedence.

`alt` `none` `str` Default: `none`

Alternative text for the image, for accessibility.

5 Diagram types

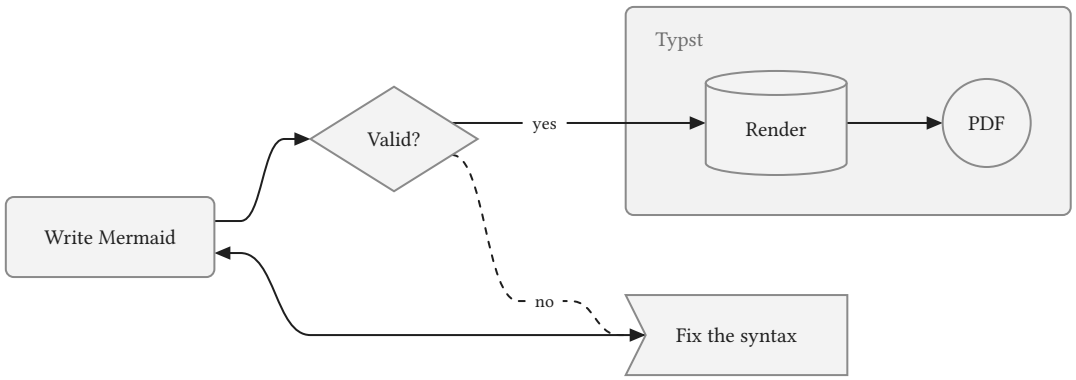
The diagram type is chosen by the source's first line. Each example below shows the source on the left, or below, and what it draws.

5.1 Flow and charts

`flowchart`

Nodes in a dozen shapes, joined by solid, dotted or thick links, in subgraphs.

Header: `flowchart`, `graph`, `flowchart-elk`



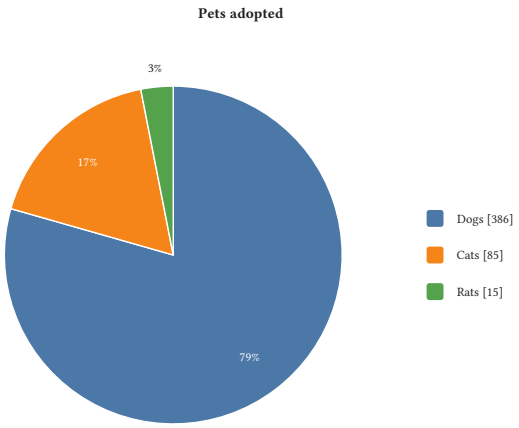

```
flowchart LR
  A[Write Mermaid] --> B{Valid?}
  B -->|yes| C[(Render)]
  B -.->|no| D>Fix the syntax]
  D --> A
  subgraph Typst
    C --> E((PDF))
  end
```

`pie`

Proportions of a whole, optionally as a donut.

Header: `pie`

```
pie showData
  title Pets adopted
  "Dogs" : 386
  "Cats" : 85
  "Rats" : 15
```



`info`

The version of pele this package is compatible with.

Header: `info`

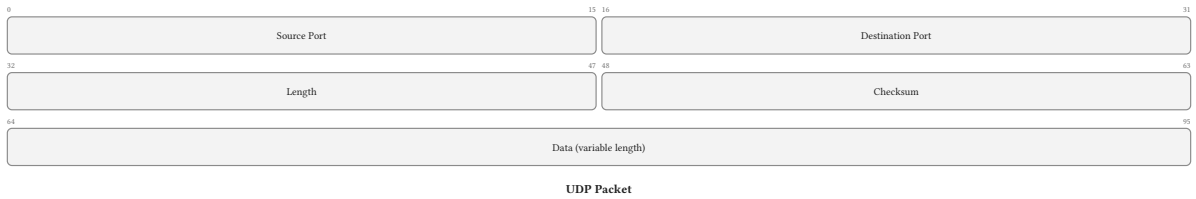
```
info
```

Pele v1.12.0

`packet`

The bit layout of a network packet or binary format.

Header: `packet`, `packet-beta`




```
packet
title UDP Packet
+16: "Source Port"
+16: "Destination Port"
32-47: "Length"
48-63: "Checksum"
64-95: "Data (variable length)"
```

`quadrantChart`

Points placed on two axes, in four labelled quadrants.

Header: `quadrantChart`

```
quadrantChart
  title Diagram choice
  x-axis Simple --> Detailed
  y-axis Static --> Dynamic
  quadrant-1 Sequence
  quadrant-2 Timeline
  quadrant-3 Pie
  quadrant-4 Class
  Flowchart: [0.45, 0.6]
  Gantt: [0.7, 0.75]
  Mindmap: [0.3, 0.25]
```



`radar`

Several series compared on radial axes.

Header: `radar-beta`

```
radar-beta
  title Restaurants
  axis food["Food"], service["Service"]
  axis price["Price"], mood["Ambiance"]
  curve a["Bistro"]{4, 3, 2, 4}
  curve b["Diner"]{3, 4, 3, 3}
  max 5
```

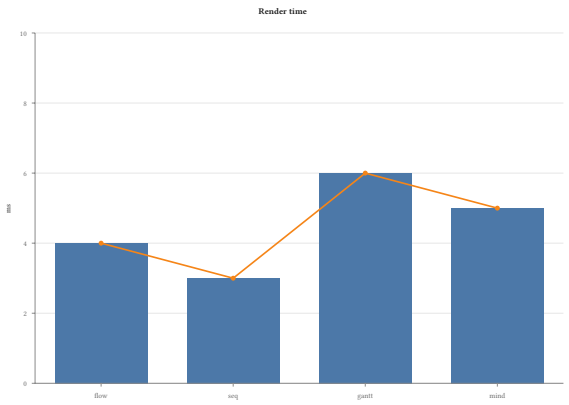


`xychart`

Bar and line series on categorical or numeric axes.

Header: `xychart`, `xychart-beta`

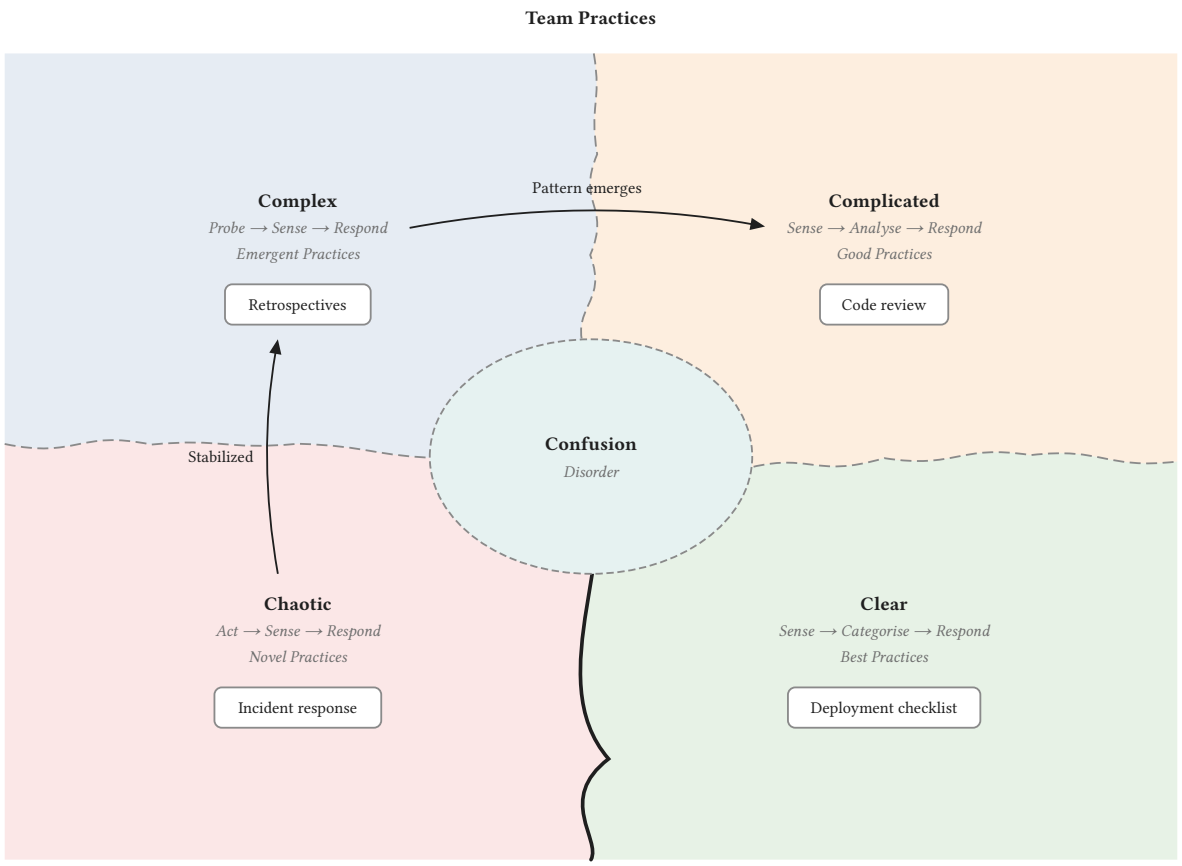
```
xychart
  title "Render time"
  x-axis [flow, seq, gantt, mind]
  y-axis "ms" 0 --> 10
  bar [4, 3, 6, 5]
  line [4, 3, 6, 5]
```



`cynefin`

The Cynefin sense-making framework, with items and transitions.

Header: `cynefin-beta`



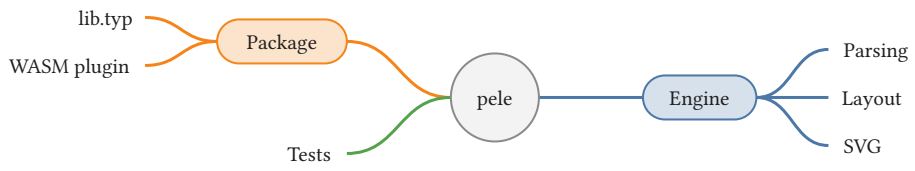

```
cynefin-beta
  title Team Practices
  complex
    "Retrospectives"
  complicated
    "Code review"
  clear
    "Deployment checklist"
  chaotic
    "Incident response"
  complex --> complicated : "Pattern emerges"
  chaotic --> complex : "Stabilized"
```

5.2 Structured layouts

`mindmap`

A tree of ideas around a central topic, given by indentation.

Header: `mindmap`

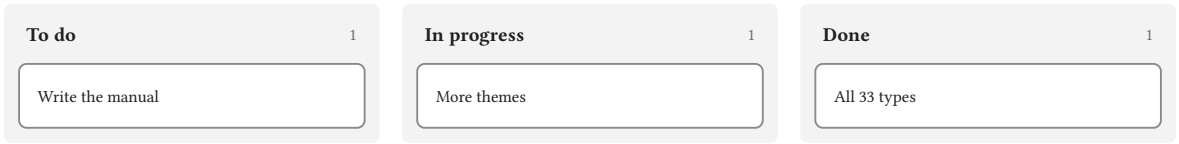



```
mindmap
  root((pele))
    Engine
      Parsing
      Layout
      SVG
    Package
      lib.typ
      WASM plugin
    Tests
```

`kanban`

Columns of task cards.

Header: `kanban`




```
kanban
  todo[To do]
    docs[Write the manual]
  doing[In progress]
    themes[More themes]
  done[Done]
    all[All 33 types]
```

`treemap`

Nested rectangles sized by value.

Header: `treemap`, `treemap-beta`

```
treemap-beta
"Budget"
  "Operations"
    "Salaries": 70
    "Equipment": 20
  "Marketing"
    "Ads": 40
    "Events": 10
```

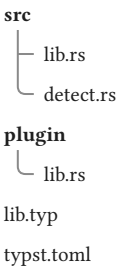


`treeView`

A file or folder hierarchy.

Header: `treeView-beta`

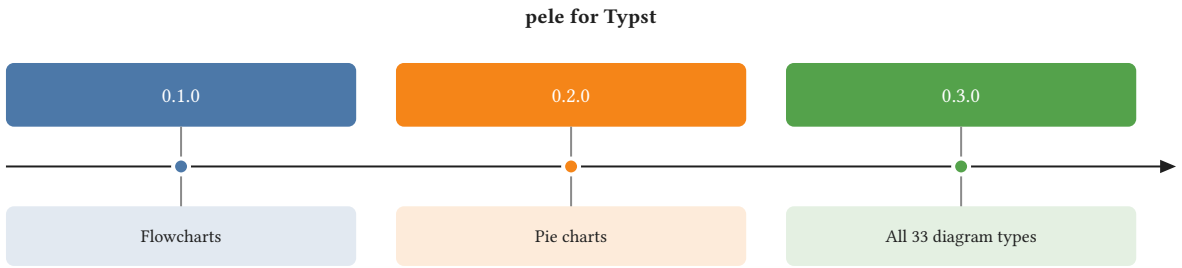
```
treeView-beta
  src/
    lib.rs
    detect.rs
  plugin/
    lib.rs
  lib.typ
  typst.toml
```



`timeline`

Events grouped by period.

Header: `timeline`

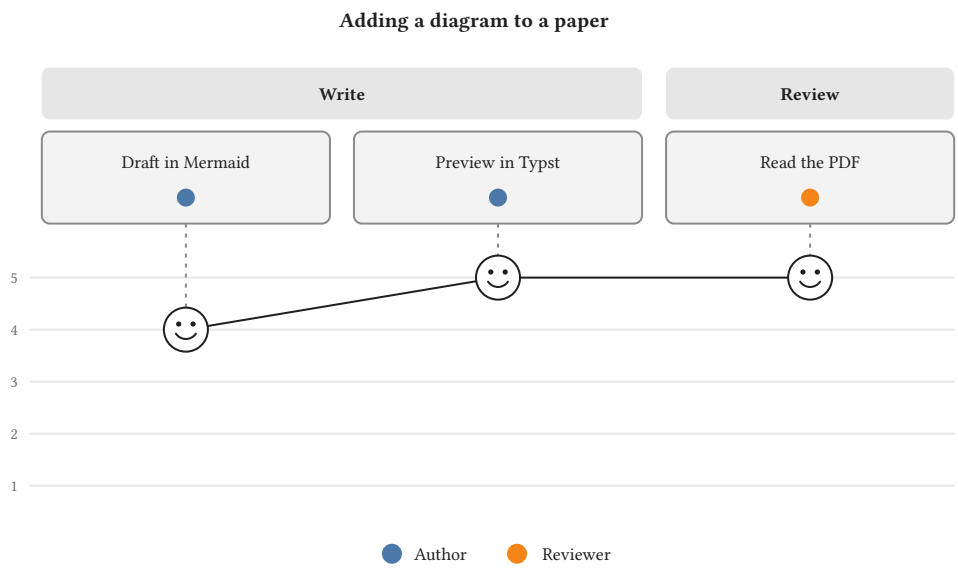



```
timeline
  title pele for Typst
  0.1.0 : Flowcharts
  0.2.0 : Pie charts
  0.3.0 : All 33 diagram types
```

`journey`

The steps of a user journey, scored by satisfaction.

Header: `journey`

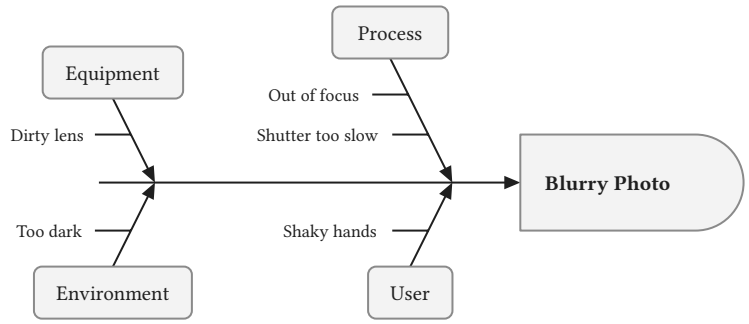



```
journey
  title Adding a diagram to a paper
  section Write
    Draft in Mermaid: 4: Author
    Preview in Typst: 5: Author
  section Review
    Read the PDF: 5: Reviewer
```

`ishikawa`

A fishbone diagram of the causes of an effect.

Header: `ishikawa`, `ishikawa-beta`

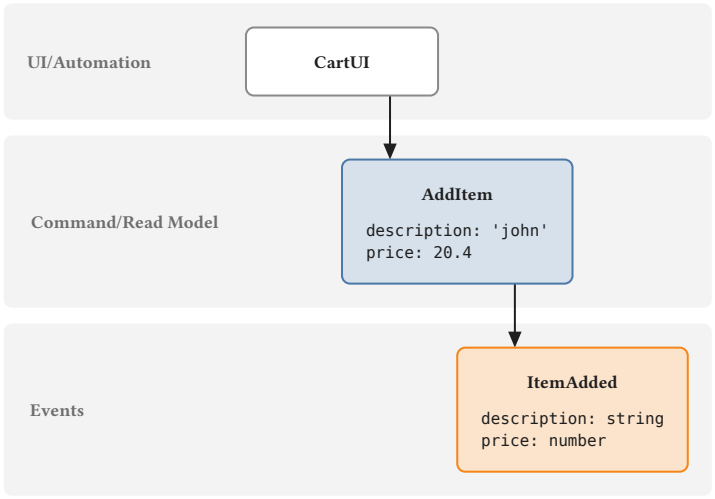



```
ishikawa-beta
  Blurry Photo
  Process
    Out of focus
    Shutter too slow
  User
    Shaky hands
  Equipment
    Dirty lens
  Environment
    Too dark
```

`eventmodeling`

Screens, commands and events laid out on a timeline.

Header: `eventmodeling`




```
eventmodeling

tf 01 ui CartUI
tf 02 cmd AddItem [[AddItem]]
tf 03 evt ItemAdded [[ItemAdded]]

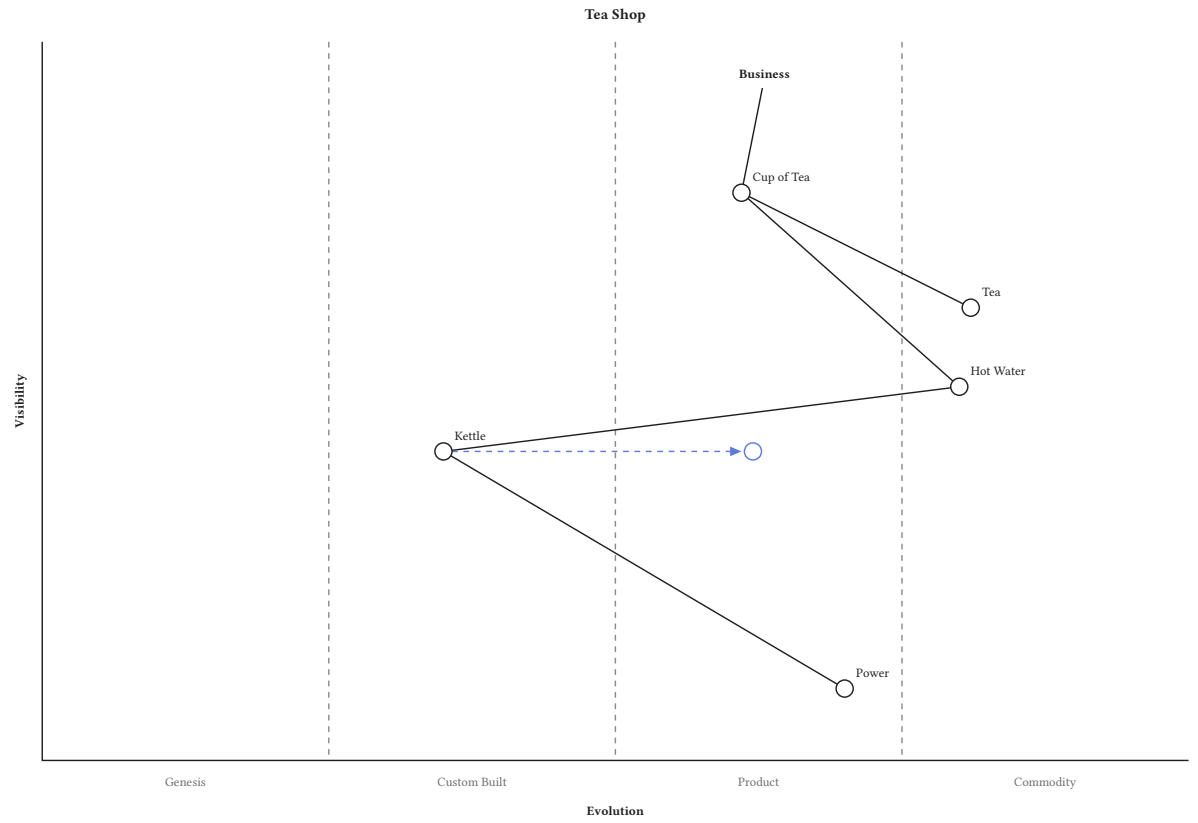
data AddItem {
  description: 'john'
  price: 20.4
}

data ItemAdded {
  description: string
  price: number
}
```

`wardley`

A Wardley map of a value chain against evolution.

Header: `wardley-beta`




```
wardley-beta
title Tea Shop

anchor Business [0.95, 0.63]
component Cup of Tea [0.79, 0.61]
component Tea [0.63, 0.81]
component Hot Water [0.52, 0.80]
component Kettle [0.43, 0.35]
component Power [0.10, 0.70]

Business -> Cup of Tea
Cup of Tea -> Tea
Cup of Tea -> Hot Water
Hot Water -> Kettle
Kettle -> Power

evolve Kettle 0.62
```

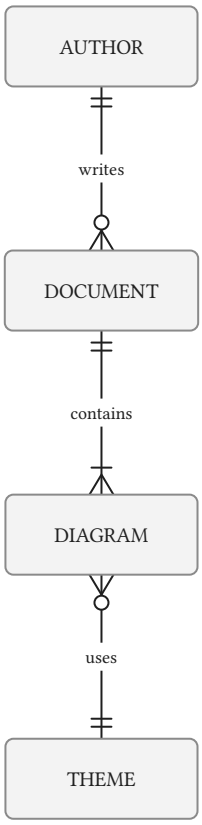
5.3 Graphs

`er`

Entities and the cardinality of their relationships.

Header: `erDiagram`

```
erDiagram
  AUTHOR ||--o{ DOCUMENT : writes
  DOCUMENT ||--|{ DIAGRAM : contains
  DIAGRAM }o--|| THEME : uses
```

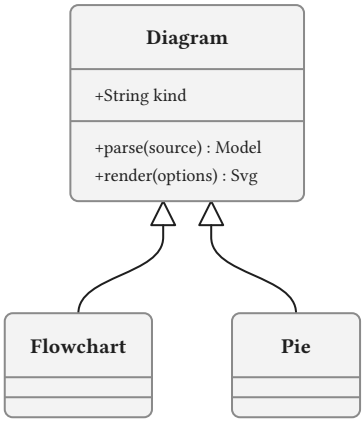


`class`

UML classes, their members and relationships.

Header: `classDiagram`, `classDiagram-v2`

```
classDiagram
  class Diagram {
    +String kind
    +parse(source) Model
    +render(options) Svg
  }
  Diagram <|-- Flowchart
  Diagram <|-- Pie
```

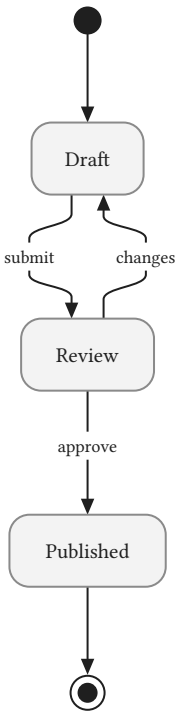


`state`

UML state machines, with composite states and notes.

Header: `stateDiagram`, `stateDiagram-v2`

```
stateDiagram-v2
  [*] --> Draft
  Draft --> Review: submit
  Review --> Draft: changes
  Review --> Published: approve
  Published --> [*]
```

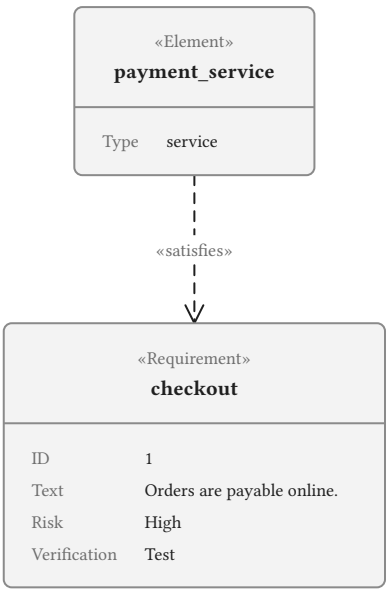


`requirement`

SysML requirements and the elements that satisfy them.

Header: `requirementDiagram`

```
requirementDiagram
  requirement checkout {
    id: 1
    text: Orders are payable online.
    risk: high
    verifymethod: test
  }
  element payment_service {
    type: service
  }
  payment_service - satisfies -> checkout
```

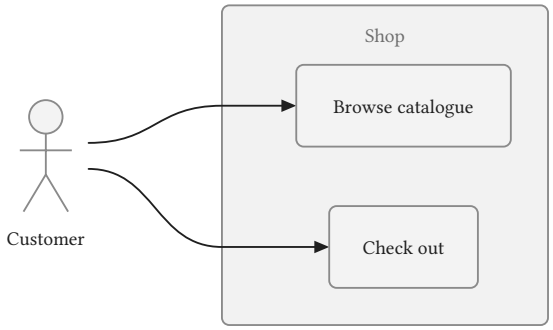


`usecase`

UML use cases, actors and system boundaries.

Header: `usecase-beta`

```
usecase-beta
actor Customer
systemBoundary Shop
  Browse[Browse catalogue]
  Checkout[Check out]
end
Customer --> Browse
Customer --> Checkout
```

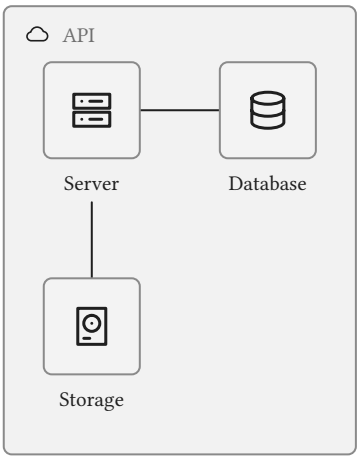


`architecture`

Services and resources in groups, joined edge to edge.

Header: `architecture-beta`

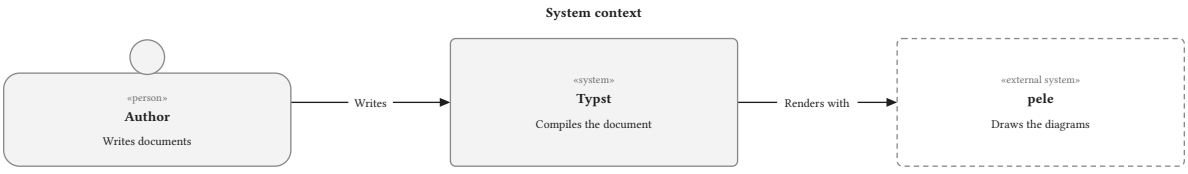
```
architecture-beta
  group api(cloud)[API]
  service db(database)[Database] in api
  service disk(disk)[Storage] in api
  service server(server)[Server] in api
  db:L -- R:server
  disk:T -- B:server
```



`c4`

C4 model diagrams of software architecture.

Header: `C4Context`, `C4Container`, `C4Component`, `C4Dynamic`, `C4Deployment`

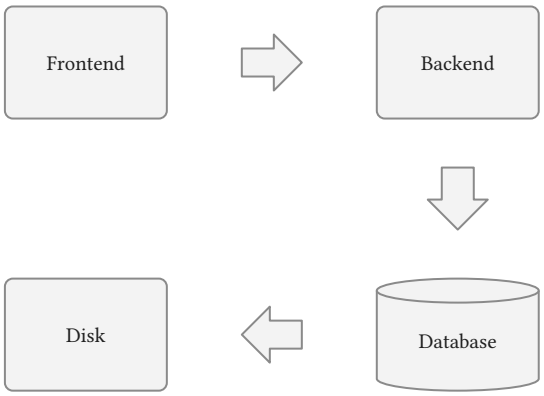



```
C4Context
  title System context
  Person(user, "Author", "Writes documents")
  System(typst, "Typst", "Compiles the document")
  System_Ext(pele, "pele", "Draws the diagrams")
  Rel(user, typst, "Writes")
  Rel(typst, pele, "Renders with")
```

`block`

Blocks placed on a grid of columns, under your control.

Header: `block`, `block-beta`

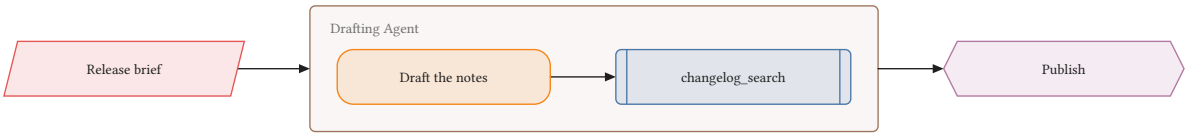



```
block
  columns 3
  Frontend arrow<[" "]>(right) Backend
  space:2 down<[" "]>(down)
  Disk left<[" "]>(left) Database[("Database")]
```

`agentflow`

AI agent workflows: tasks, tools, decisions and the agents running them.

Header: `agentflow-beta`

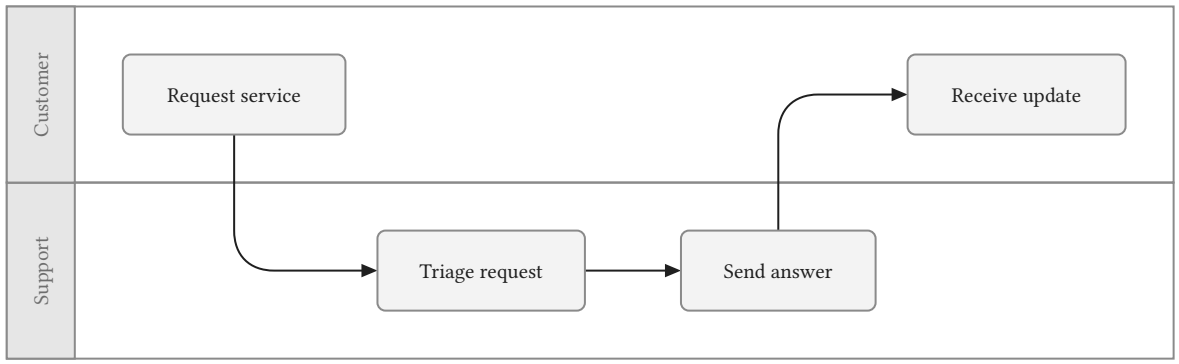



```
agentflow-beta LR
  brief["Release brief"]@{ shape: input }
  flow writer["Drafting Agent"]
    draft["Draft the notes"]@{ shape: task }
    lookup["changelog_search"]@{ shape: tool }
    draft --> lookup
  end
  publish["Publish"]@{ shape: action }
  brief --> writer
  writer --> publish
```

`swimlane`

A process split into lanes, one per participant.

Header: `swimlane-beta`



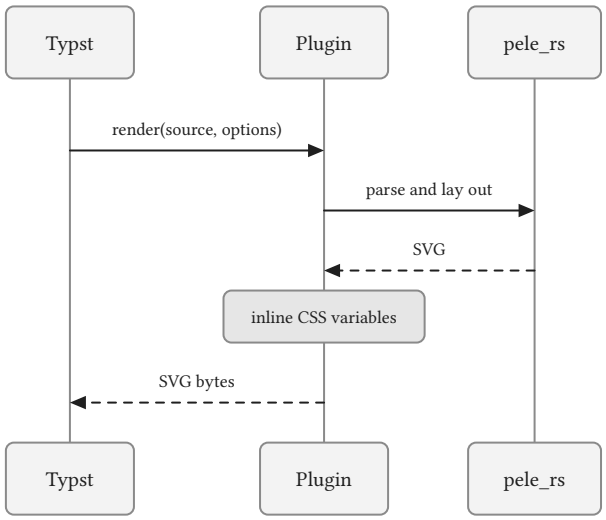

```
swimlane-beta LR
  subgraph Customer
    request[Request service]
    receive[Receive update]
  end
  subgraph Support
    triage[Triage request]
    answer[Send answer]
  end
  request --> triage
  triage --> answer
  answer --> receive
```

5.4 Specialized engines

`sequence`

Messages exchanged between participants over time.

Header: `sequenceDiagram`

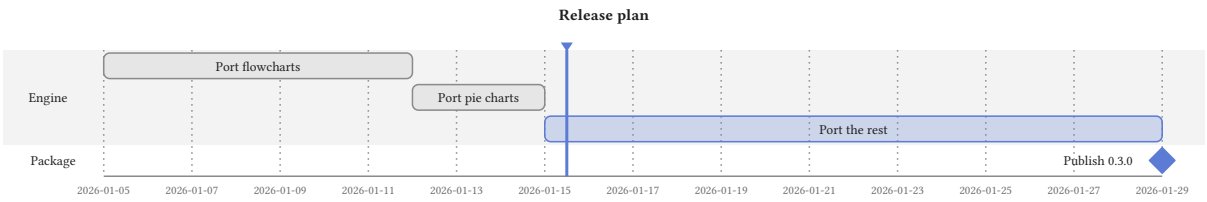



```
sequenceDiagram
  participant T as Typst
  participant P as Plugin
  participant R as pele_rs
  T->>P: render(source, options)
  P->>R: parse and lay out
  R-->>P: SVG
  Note over P: inline CSS variables
  P-->>T: SVG bytes
```

`gantt`

Tasks scheduled on a calendar. Requires `now`.

Header: `gantt`

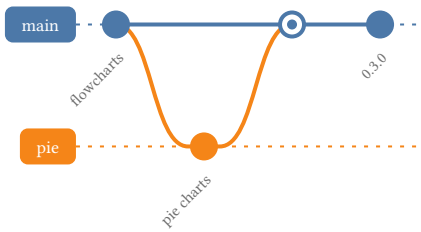



```
gantt
  title Release plan
  dateFormat YYYY-MM-DD
  section Engine
    Port flowcharts :done, f, 2026-01-05, 7d
    Port pie charts :done, p, after f, 3d
    Port the rest   :active, r, after p, 14d
  section Package
    Publish 0.3.0   :milestone, after r, 0d
```

`gitGraph`

Commits, branches and merges of a Git history.

Header: `gitGraph`



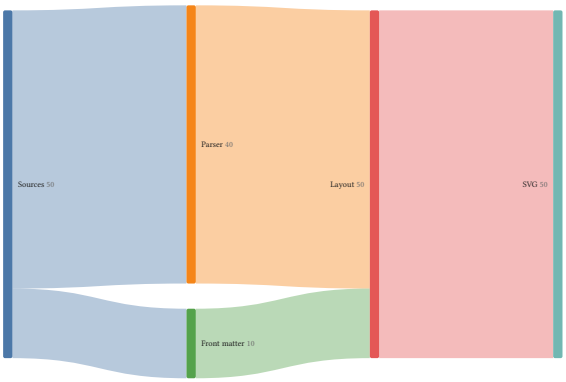

```
gitGraph
  commit id: "flowcharts"
  branch pie
  commit id: "pie charts"
  checkout main
  merge pie
  commit id: "0.3.0"
```

`sankey`

Flows between nodes, drawn with proportional widths.

Header: `sankey`, `sankey-beta`

```
sankey
Sources,Parser,40
Sources,Front matter,10
Parser,Layout,40
Front matter,Layout,10
Layout,SVG,50
```

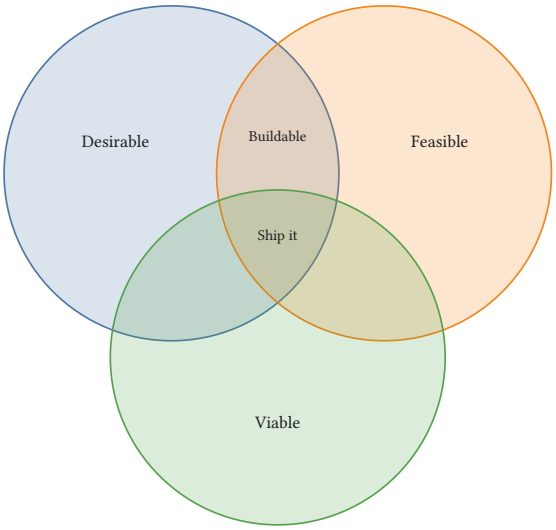


`venn`

Overlapping sets and their unions.

Header: `venn-beta`

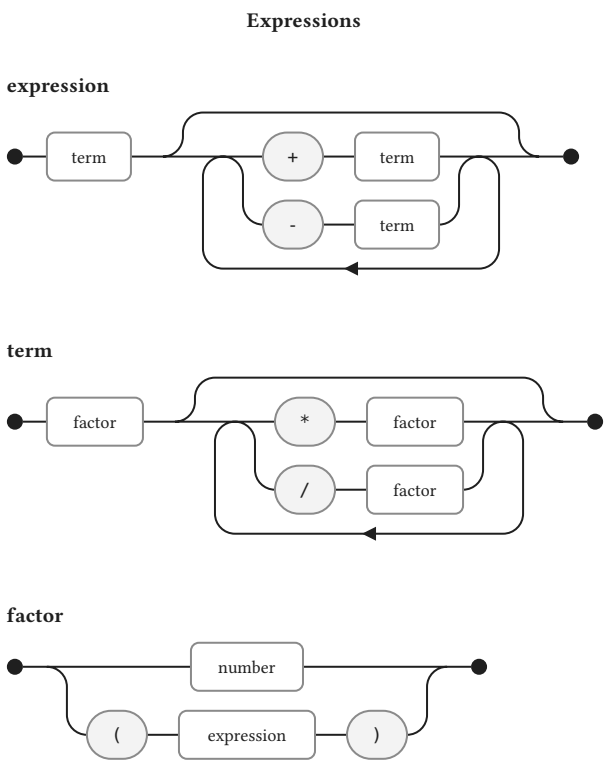
```
venn-beta
  set Desirable
  set Feasible
  set Viable
  union Desirable,Feasible["Buildable"]
  union Desirable,Feasible,Viable["Ship it"]
```



`railroad`

Syntax diagrams of a grammar, in EBNF, ABNF, PEG or pele's own notation.

Header: `railroad-beta`, `railroad-ebnf-beta`, `railroad-abnf-beta`, `railroad-peg-beta`




```
railroad-ebnf-beta
title "Expressions"
expression = term ( "+" term | "-" term )* ;
term = factor ( "*" factor | "/" factor )* ;
factor = number | "(" expression ")" ;
```

6 Compatibility

`pele_rs` 0.3.0 implements the engine of pele `1.12.0`, pinned to commit `36e584d`. Given the same source, configuration, text measurement, clock and time zone, it produces the same SVG, dimensions, links and diagnostics as the original. Its tests compare complete output for all 522 examples of upstream's documentation, among thousands of other cases.

What a browser adds to pele is outside this package: mounting into a page, measuring text in the DOM, zooming, clickable links and lazy loading. Typst's SVG renderer does not read CSS, so the plugin resolves pele's CSS variables, `color-mix()` and `calc()` into plain SVG attributes.

The full contract, including the Rust API and the test suite, is described in COMPATIBILITY.md.

pele and `pele_rs` are MIT licensed.